

Noise-Aware Geometry-Guided Graph Neural Initialization for Variational Quantum Eigensolvers

Second Review

R.K. Barath (3122235001029)

Nithyasree.K (3122235001092)

Madhangi Karimanal (3122235001073)

Guided by: Dr. Sneha Nandhini

Department of Computer Science and Engineering
Sri Sivasubramaniya Nadar College of Engineering

Academic Year 2026–2027

First Review – Queries and Answers

Q1: Run the paper's vanilla VQE under the same conditions as our method

Re-implemented VQEzy's method (Adam, $\eta = 10^{-3}$, 2000 steps) on the **same** 158 test instances, ansatz, simulator, ground energies and **same random starts** as the QNGD and GNN runs; identical device-level circuit counting (parameter-shift cost charged). Found: VQEzy's stored runs are 1-layer, so the method was re-run, not copied.

Q2: Define a proper evaluation metric

Circuit executions to target (within 1% of best energy reached; also 0.05 of exact ground) · **circuit reduction** vs. baseline = $1 - \sum \text{cost}_{\text{method}} / \sum \text{cost}_{\text{baseline}}$ (unreached runs charged full budget) · **success rate** (how often the target is reached) · **final energy gap** · all noiseless *and* noisy.

Problem Statement

Problem

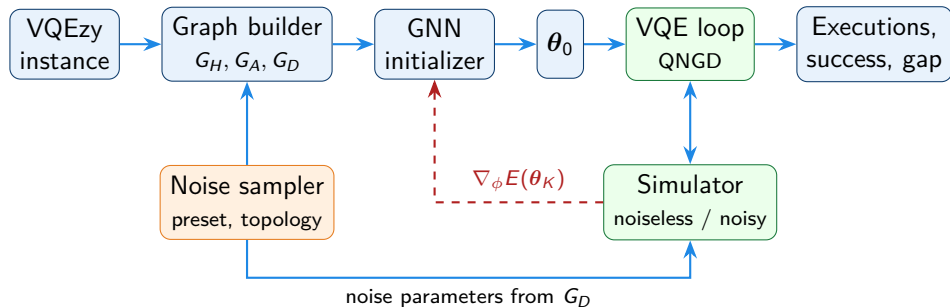
VQE on NISQ hardware is limited by the **number of quantum circuit executions**; random starting parameters waste executions and stall in flat regions.

- ▶ **Learn** $f_\phi(G_H, G_A, G_D) \rightarrow \theta_0$ from the Hamiltonian, ansatz and hardware/noise graphs
- ▶ **Goal:** VQE+QNGD from θ_0 reaches the ground state with **fewer circuit executions** than from random starts
- ▶ **Constraints:** no optimized-parameter labels; generalize to unseen Hamiltonians and an unseen qubit count (train ≤ 8 , test 12)
- ▶ **Must hold** with and without device noise

Project Objectives

- ① Reproduce the dataset's **vanilla VQE** and establish **VQE+QNGD** as the baseline
- ② Build a **three-graph GNN initializer** (Hamiltonian, ansatz, hardware/noise)
- ③ Train it **label-free by unrolling the optimizer** (GD or QNGD)
- ④ Build a **graph-driven noise model**; train noise-unaware and noise-aware variants
- ⑤ Compare all variants on held-out instances, **noiseless and noisy**

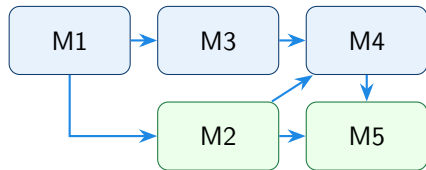
Proposed System Architecture



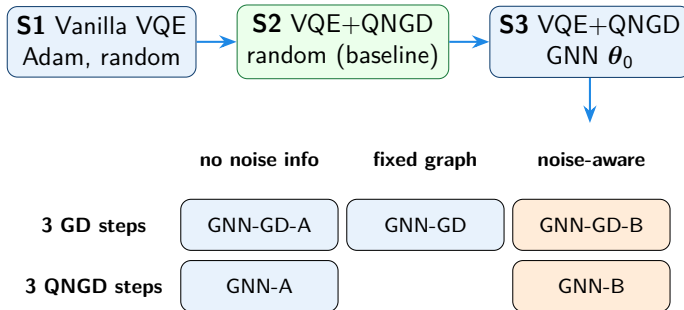
Dashed red: label-free training signal, the gradient of the energy after K unrolled optimizer steps.

System Modules

- 1 **M1 Data & Graphs:** VQEzy $\rightarrow$ G_H, G_A, G_D ; fixed splits; noise-varied copy
- 2 **M2 Simulation & Noise:** PennyLane + own GPU density-matrix simulator
- 3 **M3 GNN Initializer:** 3 GAT encoders, cross-attention, decoder
- 4 **M4 Label-free Training:** unrolled GD / QNGD objectives
- 5 **M5 VQE & Evaluation:** Adam, QNGD, circuit accounting, scoring



Methodology – the Study Ladder



Evaluation Metrics

- ▶ Circuit executions to target (device-level, `qml.Tracker`)
- ▶ Success rate
- ▶ Final energy gap
- ▶ All of the above under noise

158 held-out instances, identical random starts and accounting for every study.

Core Algorithm – Label-free Unrolled Training

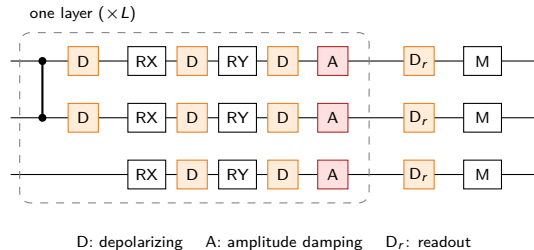
Algorithm: train f_ϕ through the optimizer

- 1 Build G_H, G_A, G_D for a training instance; predict $\theta_0 = f_\phi(G_H, G_A, G_D)$
- 2 Simulate $K = 3$ optimizer steps (noisy if noise-aware): $\theta_{k+1} = \theta_k - \eta G_k \nabla E(\theta_k)$,
$$G_k = \begin{cases} I & \text{GD, } \eta = 0.1 \\ F(\theta_k)^+ & \text{QNGD, } \eta = 0.02 \end{cases}$$
- 3 Loss $\mathcal{L} = E(\theta_K)$; back-propagate through every step (incl. F^+)
- 4 Update ϕ with Adam; repeat over instances and epochs

MAML-style objective: the start is judged by *where the optimizer gets to*, not by its own energy (minimizing $E(\theta_0)$ gave flat, slow starts).

Graph-driven Noise Model

- ▶ Noise numbers come **from** G_D **itself** $\Rightarrow$ input and simulated noise agree
- ▶ CZ: depolarizing $p_{cz} = 1 - (1 - \bar{p})^{1+6(d-1)}$ (SWAP routing on the coupling map)
- ▶ RX/RY: depolarizing, per-qubit error
- ▶ Each layer: amplitude damping from T_1
- ▶ Readout: equivalent depolarizing $p = 1.5r$ (exact for Pauli terms)
- ▶ Presets low / medium / high $\times$ topologies linear / ring / grid



Implementation Snapshot – Differentiable QNGD Simulator

```
1 def qngd_unroll(self, theta0, K, eta):
2     theta, energies = theta0, []
3     for _ in range(K):
4         e, pre = self.run(theta) #
5             density matrices
6         energies.append(e)
7         g = autograd.grad(e.sum(), theta,
8                             create_graph=
9                             True)[0]
10         F = self.metric_tensor(pre) #
11             block-diag
12         theta = theta - eta * (pinv(F) @ g
13                                )
14     energies.append(self.energy(theta))
15     return stack(energies, 1), theta
```

Validated against PennyLane

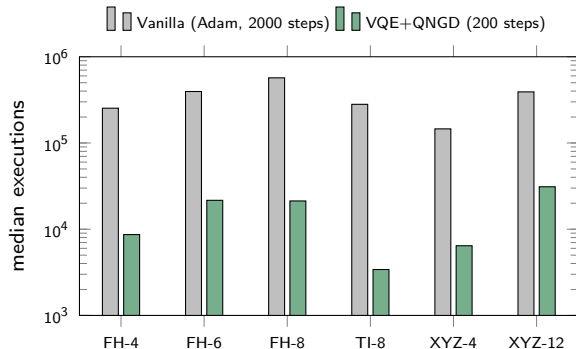
Noiseless energy	4×10^{-16}
Noisy energy (default.mixed)	9×10^{-12}
Noisy QNGD steps	1×10^{-13}
Meta-gradient vs. finite diff.	3×10^{-8}

Batched on an RTX A5000: noisy training $\approx 5\times$ faster than the earlier CPU pipeline.

Implementation Status

Modules	Status	Evidence / Remarks
M1 Data & graphs	Completed	1000/150/158 split; noise-varied copy (9 preset $\times$ topology)
M2 Simulation & noise	Completed	GPU density-matrix simulator, validated to 10^{-12}
M3 GNN initializer	Completed	3 encoders + cross-attention; with/without noise branch
M4 Training	Completed	5 variants (GD / QNGD $\times$ noise awareness) + pilot
M5 Evaluation	Completed	S1, S2, S3 noiseless and noisy
Geometry head $\hat{F}_0$, QAOA ablation	Planned	After seed study

Results 1 – Vanilla VQE vs. VQE+QNGD (same random starts)



Key Findings

- ▶ Success: **54% → 82%**
- ▶ Executions when both succeed: **$\approx 2\%$** of vanilla
- ▶ Total executions: **94% fewer**
- ▶ QNGD cheaper in **387/407** paired runs

Vanilla uses VQEzy's Adam ($\eta = 10^{-3}$), charged parameter-shift cost per step.

Results 2 – GNN Initialization on Top of QNGD (Noiseless)

Circuit executions saved vs. VQE+QNGD from random starts (158 test instances)

Group	GNN-GD	GNN-GD-A	GNN-GD-B	GNN-A	GNN-B
FH-4	+15.4%	−38.9%	−65.3%	−26.8%	−67.5%
FH-8	−7.6%	−71.8%	−68.3%	−64.9%	−77.7%
TI-8	+65.1%	+53.2%	−34.1%	+8.5%	−8.3%
XYZ-12 (unseen)	+24.3%	+35.3%	+41.6%	+47.5%	+7.7%
All	+5.8%	−29.7%	−43.7%	−22.8%	−44.8%

- ▶ Only **GNN-GD** beats the baseline without noise; all variants **generalize to 12 qubits**
- ▶ Fermi–Hubbard (FH) is hard for every initializer

Results 3 – Under Device Noise

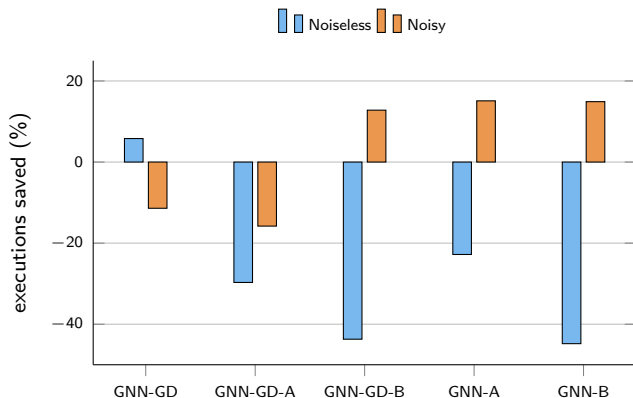
Executions saved vs. random (150 noisy instances)

Model	All	High noise	Ising
GNN-GD	−11.4%	−6.8%	0/30 reached
GNN-GD-A	−15.8%	−7.0%	0/30 reached
GNN-GD-B	+12.8%	+13.8%	+92%
GNN-A	+15.1%	+13.3%	+92%
GNN-B	+14.9%	+19.8%	+94%

Key Findings

- ▶ Noiseless-trained GD models are **noise-blind** (output changes $< 10^{-3}$ rad with G_D)
- ▶ **Training under noise** turns GD recipe from **−15.8%** into **+12.8%** (wins 70 vs. 16 instances)
- ▶ QNGD-trained models are robust even without noise training
- ▶ Noise awareness pays off most at **high noise** (GNN-B **+19.8%**)

Results Summary – No Single Winner Yet



Interpretation

- ▶ The **optimizer** gives the big win (94%)
- ▶ The **training objective** decides what a “good start” is
- ▶ 3-step QNGD training $\Rightarrow$ fast early drop, then stall (*short-horizon bias*)
- ▶ Best model depends on the **noise regime**

Limitations and Next Steps

Limitations

- ▶ One training seed per model; gaps of a few % not significant
- ▶ GD replication gap: GNN-GD-A -29.7% vs. GNN-GD $+5.8\%$ (noiseless)
- ▶ Noise-aware models never saw a zero-noise graph
- ▶ Synthetic noise presets; relative energy target
- ▶ VQEzy's stored runs are 1-layer (found in this work)

Next Steps

- ① Seed study: 3 seeds per model, best + last epoch
- ② Add a zero-noise preset to noise-aware training
- ③ Normalized longer-horizon objective
- ④ Geometry head $\hat{F}_0$ to cut QNGD's metric cost
- ⑤ QAOA ablation and Qracle-style baseline

Team Responsibilities

Team Member	Responsibilities	Current Status
R.K. Barath	[Responsibilities]	[Status]
Nithyasree.K	[Responsibilities]	[Status]
Madhangi Karimanal	[Responsibilities]	[Status]

References I

-  A. Peruzzo *et al.*, “A variational eigenvalue solver on a photonic quantum processor,” *Nature Communications*, vol. 5, 4213, 2014. DOI: 10.1038/ncomms5213.
-  J. Stokes, J. Izaac, N. Killoran, and G. Carleo, “Quantum natural gradient,” *Quantum*, vol. 4, p. 269, 2020. DOI: 10.22331/q-2020-05-25-269.
-  C. Zhang, L. Jiang, and F. Chen, “Qracle: A graph-neural-network-based parameter initializer for variational quantum eigensolvers,” *Proc. IEEE QCE*, 2025, pp. 197–207.
-  “VQEzy: A dataset for variational quantum eigensolver parameter initialization,” arXiv:2509.17322, 2025.
-  A. Cervera-Lierta, J. S. Kottmann, and A. Aspuru-Guzik, “Meta-variational quantum eigensolver,” *PRX Quantum*, vol. 2, 020329, 2021. DOI: 10.1103/PRXQuantum.2.020329.
-  C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” *ICML*, 2017. arXiv:1703.03400.

References II

-  Y. Wu, M. Ren, R. Liao, and R. Grosse, “Understanding short-horizon bias in stochastic meta-optimization,” *ICLR*, 2018. arXiv:1803.02021.
-  J. R. McClean *et al.*, “Barren plateaus in quantum neural network training landscapes,” *Nature Communications*, vol. 9, 4812, 2018. DOI: 10.1038/s41467-018-07090-4.
-  P. Veličković *et al.*, “Graph attention networks,” *ICLR*, 2018. arXiv:1710.10903.
-  V. Bergholm *et al.*, “PennyLane: Automatic differentiation of hybrid quantum-classical computations,” arXiv:1811.04968, 2018.

Thank You

Questions and Discussion